Harnessing AI Agents

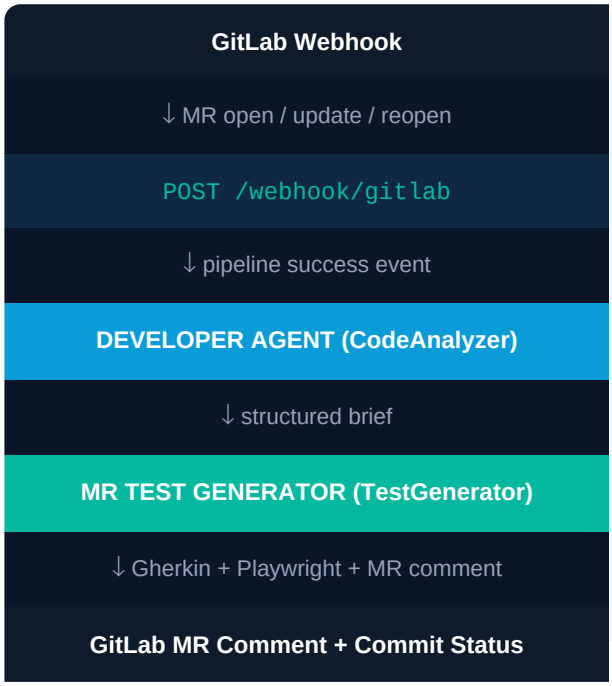
Profiles · Rules · Skills

A practical guide to designing multi-agent pipelines
that react to GitLab events and generate tests automatically.

Based on the **quality** project — a production GitLab webhook app
that uses Claude to analyse MRs and generate BDD + Playwright tests.

1. System Architecture

The quality project is a FastAPI application deployed as a Docker container. GitLab sends webhook payloads to its single endpoint. The app dispatches work to two specialised AI agents running in background tasks — no blocking, no polling.



Key design decisions

- Two agents, clear separation of concerns: one analyses, one generates.
- MR webhook fires immediately → sets *pending* commit status → visible in the MR.
- Pipeline success event triggers generation — agents never run on broken code.
- All generation runs in BackgroundTasks — webhook always returns instantly.
- Dedup via _done + _processing sets prevents double-runs on retries.
- Progressive MR comment: posted immediately, edited in-place at each step.

2. Webhooks vs GitLab Apps — Why Webhooks Win

GitLab offers two ways to integrate external tooling: **webhook payloads** (GitLab pushes events to your server) and **GitLab Apps / OAuth integrations** (your app pulls data using OAuth tokens). The quality project is built on webhooks. This section explains why — and when that choice matters most for AI agent pipelines.

Dimension	Webhooks	GitLab Apps / OAuth
Authentication	Single shared secret in the X-Gitlab-Token header. Set once, never rotates unless you choose to.	OAuth 2.0: client_id, client_secret, authorization code, access token, refresh token — all managed by you.
Token lifecycle	No tokens. The secret never expires.	Access tokens expire (typically 2h). Refresh tokens must be stored, rotated, and reissued automatically.
Local development	ngrok tunnel or any HTTP forwarder. One command: ngrok http 8000	Requires a registered redirect URI. Local dev needs a registered app with gitlab.com or your instance.
Deployment	Any server that can receive HTTP POST requests. Docker, Render, Azure Container Apps, even a VPS.	Must be reachable by GitLab for the OAuth callback. Additional infra for token storage (DB or secrets manager).
Scope of access	Scoped by what GitLab sends in the payload. You request only what you need via the API key.	Scopes negotiated at install time and stored in the token. Mismatched scopes break the integration silently.
Multi-repo support	One webhook endpoint, configured per project in GitLab. Each project sends to the same URL.	App must be installed per group or per project. Installation state must be tracked.
Debugging	GitLab shows every webhook delivery in the UI with the full payload, response code, and retry history.	OAuth errors surface as 401s at call time — no central delivery log, harder to trace which call failed.
Portability	Identical pattern on GitHub, Bitbucket, Azure DevOps. Migrating CI provider = update one URL.	Each platform has its own OAuth implementation. Migration requires rewriting the auth layer.
Security surface	One secret to protect. Rotate it in GitLab settings and update one env var.	client_secret + per-installation tokens. Compromise of any token may expose all projects.

GitLab instance support	Works on gitlab.com and any self-hosted instance without configuration changes.	Self-hosted instances may have OAuth disabled or require admin-level app registration.

2.1 The operational reality for AI agent pipelines

AI agents add a new dimension to the webhook vs. app decision: **latency and reliability matter more than feature richness**. Here is what that means in practice for the quality project:

Token expiry kills long-running agents

The quality app's process_mr() function can take 20-40 seconds. An OAuth access token that expires mid-run causes a 401 with no retry — the agent silently fails and leaves the MR in 'running' status forever. Webhooks have no tokens to expire.

Webhook delivery logs are your best debugging tool

GitLab records every webhook delivery: timestamp, payload, HTTP response, and whether it was retried. When an agent fails you can replay the exact payload from the GitLab UI. OAuth integrations have no equivalent.

The secret is your only attack surface

The quality app validates X-Gitlab-Token on every request (middleware.py). That is one secret to rotate, audit, and protect. An OAuth integration has a client_secret plus N installation tokens — each one a credential that can be leaked or expire.

Portability matters when your team grows

Today the quality project runs on GitLab. Tomorrow it might need to run on GitHub for a partner project. Webhooks are identical across providers — only the payload schema changes. OAuth is entirely provider-specific.

Fewer moving parts = fewer incidents

The quality app has zero background jobs managing token refresh, zero database tables storing OAuth state, and zero scheduled tasks rotating credentials. That is not a small thing at 2 AM when something breaks.

2.2 When to use GitLab Apps instead

Webhooks are not always the right choice. There are specific scenarios where the OAuth app model is the correct tool:

Use a GitLab App when...	Why webhooks fall short here
You need to act on user-level behalf (e.g. assign MRs, impersonate a user)	Webhooks authenticate as a project token — no user identity. OAuth gives you a token scoped to a real user.
You are building a GitLab Marketplace listing (public SaaS product)	Marketplace requires OAuth install flow and GitLab's app registration process.
You need fine-grained per-installation permissions (different scopes per group)	Webhooks use a single project token. OAuth scopes can be negotiated per installation.
Your integration is pull-based (you poll GitLab, not the other way)	Webhooks are push-only. For polling you need an API token anyway — OAuth is cleaner.

> ***Bottom line:*** *For an internal AI agent pipeline triggered by CI events — where you own the server, the team is small, and reliability matters more than user-level delegation — webhooks are strictly simpler, more debuggable, and easier to operate than OAuth apps. The quality project's entire auth layer is 12 lines of middleware.*

2.3 The quality project's webhook auth (12 lines)

The entire authentication layer for the quality project is a single FastAPI middleware that checks one header. Compare this to the ~200 lines of OAuth plumbing a GitLab App would require:

```
# app/middleware.py — the complete auth layer
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import JSONResponse
import os

GITLAB_TOKEN = os.environ['GITLAB_WEBHOOK_SECRET']

class GitlabTokenMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        if request.url.path.startswith('/webhook'):
            token = request.headers.get('X-Gitlab-Token', '')
            if token != GITLAB_TOKEN:
                return JSONResponse({'error': 'Unauthorized'}, status_code=401)
        return await call_next(request)
```

One environment variable. One header check. No token storage, no refresh jobs, no installation state. This is the entire security model.

3. Agent Profiles

A **profile** is the identity contract for an agent. It answers four questions: *Who is it? What is it for? What does it receive? What does it produce?* Profiles live in agents/<agent-name>/profile.md and are human-readable documentation — they are NOT injected into the model prompt. They are the contract that rules and skills implement.

> *Rule of thumb: if two people on the team disagree about what an agent should do, the profile is what you update — not the code. A clear profile prevents scope creep.*

2.1 DEVELOPER AGENT profile

This agent acts as a senior engineer who reads code before any tests are written. Its output is a structured brief — never test code.

Field	Value
Identity	Senior software engineer reading MR diffs
Purpose	Fill the context gap — diffs alone don't show intent
Inputs	MR title + description, unified diff, up to 10 file contents
Output	Structured brief: Change Tree, Business logic, Data flows, Integration points, Error paths, Test priorities
Constraints	Analysis only — never generates test code Caps output at ~800 words Flags uncertainty explicitly

2.2 MR TEST GENERATOR profile

This agent consumes the DEVELOPER AGENT's brief and generates test artifacts. It never analyses code directly — it trusts the brief.

Field	Value
Identity	AI code quality agent monitoring GitLab MRs
Trigger	GitLab webhook: MR open / update / reopen
Inputs	MR title + description, diff, file contents (max 10), existing test examples, DEVELOPER AGENT brief (optional)
Outputs	1. Gherkin .feature file 2. Playwright TypeScript test file 3. Formatted MR comment (live, edited in-place) 4. Commit status: pending → running → success/failed
Constraints	Max 10 files per MR, max 8 000 tokens of diff Always returns 202 immediately — never blocks the MR Posts error comment on failure — never silent

2.3 Profile template

Use this template when creating a new agent:

```
# Agent: <Name>

## Identity

One sentence: who this agent is and what perspective it brings.

## Purpose

Why does this agent exist? What problem does it solve that no existing agent covers
?

## Trigger

What event or call activates this agent?

## Inputs

- Item 1 (type, source, size limit)
- Item 2 ...

## Outputs

- Output 1 (format, destination)
- Output 2 ...

## Operating Constraints

- Hard limits (token caps, file counts, timeouts)
- What this agent NEVER does (role separation)
```

4. Agent Rules

Rules are **enforceable behavioural constraints** — things the agent must or must not do regardless of what the LLM might otherwise produce. They live in agents/<agent-name>/rules.md and are a mix of: prompt instructions (injected into the system prompt), code-level guards (checked in Python before/after calling the model), and operational policies (enforced by the pipeline, not the model).

> *A rule without a **Why** is a rule that gets removed the moment someone doesn't understand it. Always document the reason — often a past incident or a hard-learned constraint that isn't obvious from the code.*

3.1 DEVELOPER AGENT rules

R1 Analyse before generating

Always runs before the MR TEST GENERATOR. If it fails, the test generator continues without the brief — never blocks the pipeline. (code guard in main.py)

R2 Stay in analysis mode

Never produces test code, Gherkin, or Playwright. Sole output is the structured brief. Mixing roles degrades both outputs. (prompt instruction)

R3 Name names

Must reference actual identifiers from the code — function names, variable names, endpoints. Generic descriptions are not acceptable. (prompt instruction)

R4 Prioritise by risk, not by size

A one-line auth check outweighs a 200-line UI refactor. Test priorities must reflect business impact. (prompt instruction)

R5 Pass the brief as structured context

Brief injected into the test generator's system prompt under '## Code Analysis', after style examples and before the diff. (code — _build_system in test_generator.py)

R6 Cap analysis length

Output must stay under 800 words. If the MR is too large, analyse highest-risk files only and note which were skipped. (max_tokens=1024 in code_analyzer.py)

R7 Learn from the test generator's output

If generated scenarios miss a flagged edge case, that gap is noted in the agent's rules as a standing instruction. Closes the feedback loop. (operational policy)

3.2 MR TEST GENERATOR rules

R1 Learn from existing tests

Fetch up to 3 existing test files before every generation. Prepend under '## Existing test style — match this'. Missing files are not an error. (code guard)

R2 Match existing test style

Inject the fetched test files as '## Existing test style — match this'. Style consistency matters more than theoretical correctness. (code — gitlab_client.py)

R3 Surface learning signals in the comment

Every MR comment ends with a feedback line. Reactions logged to feedback-log.jsonl. Without feedback the agent cannot distinguish good from bad generations. (template)

R4 Honour explicit overrides in MR description

A fenced agent-instructions block in the MR description adds per-MR instructions to the prompt. Gives developers an escape hatch without touching any config; builds trust. (code — parse before prompt build)

R5 Never block the MR

Generation always runs in BackgroundTasks. Webhook always returns 202 immediately. On failure: post error comment, log, exit cleanly. (code — FastAPI BackgroundTasks)

R6 Respect file caps

Max 10 files, max 8 000 tokens of diff. Prefer most-changed files when truncating. Large MRs produce noisy, low-quality output. (code — _filter_relevant_files)

R7 Refine rules when patterns stabilise

After 10 positive reactions, propose an update to the agent's rules.md via a new MR. Learning should live in the repo, owned by the team. (operational policy)

R8 Version the prompt

Log system prompt hash to the HTML report footer. When rules/skills change, the hash changes — correlates output quality with prompt versions. (code — report builder)

3.3 Rule anatomy — the three-part structure

Every rule in this project follows the same structure, which makes them maintainable and actionable:

```
## R<N> — <Short imperative title>

<What the agent must do or must not do — one or two sentences.>

**Why:** <The reason — often a past incident or a hard constraint>

(Where enforced: prompt instruction | code guard | operational policy)
```

5. Agent Skills

Skills serve two purposes. The **prose section** (readable headings) documents what the agent is capable of — useful for humans reviewing the agent design. The **SKILL blocks** (HTML comments) contain the actual system prompt text that gets extracted by code and sent to the model. This dual-use design keeps prompt engineering version-controlled and auditable alongside the rest of the codebase.

> *Skills are the only place where raw prompt text lives. Keeping them in Markdown files (not in Python strings) means prompt changes are visible in git diffs — they go through code review like any other change.*

4.1 File structure

A skills file mixes two things: human-readable capability documentation and machine-readable SKILL blocks. The extraction mechanism uses HTML comments as delimiters:

```
## 2. Gherkin Generation
- Writes valid BDD .feature files from code changes and MR intent
- Covers happy path, edge cases, and error/boundary conditions
  ...
<!-- SKILL:GHERKIN_SYSTEM -->
You are an expert QA engineer specializing in BDD.
Given a GitLab MR title, description, and diff, generate Gherkin content.
Rules:
- Write realistic, concrete scenarios based on actual code changes
- Cover happy path, edge cases, and error states
  ...
<!-- END:GHERKIN_SYSTEM -->
```

4.2 How skills are extracted (the mechanism)

Both code_analyzer.py and test_generator.py use the same extraction pattern. The skill block is loaded once at module import time — not on every API call — so it is fast and fails loudly at startup if a skill is missing:

```
import re
from pathlib import Path
_SKILLS_FILE = Path(__file__).parent.parent / 'agents' / 'developer-agent' / 'skill
s.md'
def _extract_skill(name: str) -> str:
    text = _SKILLS_FILE.read_text()
    match = re.search(
        rf'<!-- SKILL:{name} -->\n(.*?)<!-- END:{name} -->',
        text, re.DOTALL
```

```
    )
    if not match:
        raise ValueError(f"Skill block '{name}' not found")
    return match.group(1).strip()

# Loaded once at module import — fails fast at startup
DEVELOPER_SYSTEM = _extract_skill('DEVELOPER_SYSTEM')
```

4.3 DEVELOPER AGENT skills

Skill	What it does
Code Intent Recognition	Reads function/variable names, comments, and structure to infer purpose. Distinguishes bug fixes from new features from refactors.
Business Logic Extraction	Detects conditional branches, guard clauses, and validation rules. Surfaces implicit business constraints easy to miss in a diff view.
Data Flow Mapping	Traces how inputs move through functions: transformations, aggregations, filtering. Identifies state reads vs. writes, and side effects.
Integration Point Detection	Recognises HTTP calls, database queries, message queue interactions, file I/O. Flags integration points that are high-risk for regressions.
Edge Case & Error Path Analysis	Identifies null/undefined handling, empty collections, zero values. Spots try/catch blocks and boundary conditions.
Test Priority Ranking	Scores each identified behaviour by risk (likelihood x impact). Calls out the single most important thing to test first.

4.4 MR TEST GENERATOR skills

Skill	What it does
Diff Analysis	Parses unified diffs, filters to relevant extensions (.ts .tsx .py .go ...), excludes test files, deleted files, and generated code.
Gherkin Generation	Writes valid BDD .feature files. Covers happy path, edge cases, error states. Uses Feature, Rule, Scenario, Scenario Outline, Examples correctly.
Playwright Generation	Maps each Gherkin scenario 1:1 to a test() block. Applies Page Object Model. Uses getByRole, getByLabel, getByTestId selectors.
GitLab Operations	Fetches MR diff and file contents via GitLab REST API. Posts/edits MR comments. Sets commit statuses (pending/running/success/failed).
Report Generation	Builds self-contained HTML report with sidebar, tabbed code view, and copy buttons. Committed to MR branch — no external hosting.
Context Awareness	Fetches up to 3 existing test files from the target repo as few-shot style examples. Injects them into the system prompt before generation.

6. How It All Connects — Prompt Assembly

Understanding how profiles, rules, skills, and runtime context combine into the final API call is the key to debugging and improving agent output.

5.1 DEVELOPER AGENT prompt assembly

The DEVELOPER AGENT has a single system prompt — the DEVELOPER_SYSTEM skill block. The user message is assembled from the MR context:

```
# system prompt = DEVELOPER_SYSTEM skill block (extracted at import time)

# user message = assembled from MR context
parts = [
    f'## MR Title\n{mr_title}',
    f'## Description\n{mr_description[:1000]}',   # R6: cap length
    f'## Changed Files\n{files_section}',          # up to 5 files, 2000 chars each
    f'## Diff\n```diff\n{diff_text[:6000]}\n```', # R6: cap diff
    'Produce the structured analysis brief:',
]

message = await client.messages.create(
    model='claude-sonnet-4-6',
    max_tokens=1024,      # R6: enforces 800-word cap
    system=DEVELOPER_SYSTEM,
    messages=[{'role': 'user', 'content': '\n\n'.join(parts)}],
)
```

5.2 MR TEST GENERATOR prompt assembly

The test generator builds a richer system prompt by layering multiple context sources in a defined order. The order matters — later sections override or refine earlier ones:

```
def _build_system(base, example_tests, code_analysis):
    parts = [
        base,                          # 1. GHERKIN_SYSTEM or PLAYWRIGHT_SYSTEM ski
ll
    ]
    if example_tests:                  # 2. R2: few-shot style examples
        parts.append('## Existing test style — match this\n...')
    if code_analysis:                  # 3. R5: DEVELOPER AGENT brief (highest prio
rity)
        parts.append(f'## Code Analysis\n{code_analysis}')
    return '\n\n'.join(parts)

# Per-MR agent-instructions block (R4) appends per-MR instructions to base
```

5.3 Context injection order (priority)

When multiple context sources are present, the model reads them in this order. Later items override earlier ones when they conflict:

Priority	Source	When present	Enforced by
1 — base	GHERKIN_SYSTEM or PLAYWRIGHT_SYSTEM skills	Always	Extracted at import time
2	Existing test files from target repo (up to 3)	If files exist	R1 — fetched before every call
3 — highest	DEVELOPER AGENT brief	If DA succeeded	R5 — injected last, read last
Override	agent-instructions block in MR description	If block present	R4 — per-MR instructions appended to base

7. Adding a New Agent — Step by Step

Use this checklist when you want to extend the pipeline with a new agent — for example, a Security Reviewer or a Documentation Writer that runs after the test generator.

Step 1 — **Create the agent directory**

```
mkdir agents/my-new-agent
touch agents/my-new-agent/profile.md
touch agents/my-new-agent/rules.md
touch agents/my-new-agent/skills.md
```

Step 2 — **Write the profile**

```
Fill profile.md with Identity, Purpose, Trigger, Inputs, Outputs, Constraints.
Keep it under one page. If you need more, the agent is doing too much.
```

Step 3 — **Write the rules**

```
Write at least: one rule for what it NEVER does (role separation),
one rule for failure behaviour (never block the pipeline),
one rule for output caps (token/file limits).
```

Step 4 — **Write the skills**

```
Prose documentation first, then add a SKILL block with the system prompt:

You are...
```

Step 5 — **Add the extractor module**

```
Create app/my_agent.py. Copy the _extract_skill pattern from code_analyzer.py.
Load the skill at module import time — not inside the async function.
```

Step 6 — **Wire it into main.py**

```
Decide where in process_mr() it runs. Add a _update() call after it completes
so the live MR comment reflects its progress.
```

Step 7 — **Write a test**

```
Add tests/test_my_agent.py with at least one happy-path test and one failure test.
Use pytest-asyncio. Mock the Anthropic client — don't call the real API in tests.
```

8. Lessons & Quick Reference

Patterns that emerged from building and operating the quality project:

✓ **Separate who analyses from who generates**
The DEVELOPER AGENT / Test Generator split means each prompt is focused. One agent doing both produces worse output at double the latency.

✓ **Never block the pipeline**
The webhook must return immediately. All work in BackgroundTasks. On any failure: post an error comment, log, exit. The team must never be silently blocked by a quality tool.

✓ **Prompt text belongs in version control**
SKILL blocks in skills.md means every prompt change is a git diff. It goes through code review. The hash in the report footer ties output quality to the exact prompt version that produced it.

✓ **Existing tests are the best style guide**
Fetching real test files from the target repo and injecting them as few-shot examples produces output that fits the codebase. A generic prompt produces generic tests that get deleted; tests that look familiar get kept and extended.

✓ **The escape hatch builds trust**
R4 (agent-instructions block in the MR description) lets developers add per-MR instructions without touching any configuration or file. Teams that trust they can steer a tool are more likely to keep using it.

✓ **Dedup is mandatory at any scale**
GitLab retries webhook delivery. Without the _done set, every retry re-runs the generation and posts a duplicate comment. The fix is one set checked before any work begins.

✓ **Progressive comments feel responsive**
Posting one comment and editing it in-place (rather than posting multiple comments) keeps the MR thread clean and gives developers live progress without notification noise.

Quick reference — file map

File	Purpose	Who reads it
`agents/<name>/profile.md`	Identity contract — who, purpose, inputs, outputs	Humans (design review)
`agents/<name>/rules.md`	Behavioural constraints — must/must not, with Why	Humans + code (enforced as guards or prom
`agents/<name>/skills.md`	Capability docs + SKILL blocks (extracted system prompts)	Code (_extract_skill) + Humans
`app/code_analyzer.py`	DEVELOPER AGENT — extracts DEVELOPER_SYSTEM, calls Claude	Python runtime
`app/test_generator.py`	Test Generator — extracts GHERKIN_SYSTEM + PLAYWRIGHT_SYSTEM	Python runtime
`app/main.py`	Pipeline orchestration — webhook routing, dedup, progress updates	Python runtime
`agents/<name>/feedback-log.jsonl`	Reaction log — thumbs up/down from MR comments	Scheduled job reads it to track generation qu

*Every agent directory follows the same three-file convention: **profile.md** (contract), **rules.md** (constraints), **skills.md** (capability + prompts). This is what makes agents reviewable, maintainable, and replaceable.*